

Tecniche avanzate

Query ricorsive

CTE che si richiamano da sole per esplorare strutture a livelli.

Qui una query costruisce il passo successivo da sola

Le query ricorsive servono quando i dati hanno una struttura a livelli. Per esempio:

- cartelle e sottocartelle
- categorie e sotto-categorie
- dipendenti e responsabili

In questi casi non basta guardare solo un livello. Devi poter salire o scendere nella gerarchia.

```
WITH RECURSIVE numeri AS (  
  SELECT 1 AS n  
  UNION ALL  
  SELECT n + 1  
  FROM numeri  
  WHERE n < 5  
)  
SELECT * FROM numeri;
```

Questo esempio non usa una gerarchia vera, ma mostra il meccanismo: la query parte da un valore iniziale e poi genera il successivo finché la condizione resta valida.

Perché si chiamano ricorsive

Si chiamano così perché **una parte della query usa il risultato generato dalla query stessa**. È un'idea potente, ma richiede ordine mentale.

Per chi inizia, la chiave è questa: c'è un punto di partenza e c'è una regola che produce il passo seguente.

Quando diventano preziose

Sono molto utili quando vuoi percorrere un albero o una gerarchia senza farlo manualmente riga per riga.

All'inizio possono sembrare avanzate, ma con la giusta immagine mentale diventano molto più comprensibili.

Set operations

UNION, UNION ALL, INTERSECT ed EXCEPT per combinare risultati di query diverse.

Qui lavori su insiemi di risultati

Le set operations permettono di combinare il risultato di più query. Invece di pensare riga per riga, qui ragioni per insiemi.

È un po' come mettere insieme due liste, tenere solo gli elementi in comune o togliere una lista dall'altra.

- **UNION** : unisce senza duplicati
- **UNION ALL** : unisce con duplicati
- **INTERSECT** : tiene solo ciò che coincide
- **EXCEPT** : toglie il secondo insieme dal primo

```
SELECT email FROM clienti_attivi
UNION
SELECT email FROM newsletter;
```

Questa query unisce le email provenienti da due fonti diverse, rimuovendo i duplicati.

La differenza più importante da ricordare

UNION e **UNION ALL** si assomigliano, ma non fanno la stessa cosa:

- **UNION** elimina i duplicati
- **UNION ALL** conserva tutto

Se vuoi tenere anche i ripetuti, scegli **UNION ALL** .

Una regola tecnica da non dimenticare

Per combinare query con queste operazioni, **le colonne restituite devono essere compatibili** per numero e tipo.

In pratica, le due query devono parlare la stessa lingua in uscita.

Window functions

ROW_NUMBER, RANK, DENSE_RANK e OVER per fare classifiche e confronti riga per riga.

Qui guardi il gruppo senza schiacciarlo in una sola riga

Le window functions sono funzioni molto utili quando vuoi analizzare un gruppo di righe, ma senza trasformarlo in una sola riga come fa **GROUP BY**.

Sono perfette per classifiche, posizioni e confronti tra righe vicine.

```
SELECT
    nome,
    totale,
    ROW_NUMBER() OVER (ORDER BY totale DESC) AS posizione
FROM ordini;
```

Questa query assegna un numero di posizione a ogni riga, ordinando i risultati per **totale** dal più grande al più piccolo.

Le funzioni più famose

Funzioni famose:

- **ROW_NUMBER()**
- **RANK()**
- **DENSE_RANK()**

Il ruolo di OVER

La parte `OVER ( . . . )` dice alla funzione **su quale finestra di righe deve lavorare** e con quale ordine.

Senza `OVER`, questa famiglia di funzioni non saprebbe su quale insieme ragionare.

Quando sono utili

Le window functions servono spesso per:

- creare classifiche
- numerare le righe
- confrontare risultati vicini
- aggiungere analisi senza perdere il dettaglio riga per riga

Sono un tema un po' più avanzato, ma molto importante quando inizi a fare analisi più ricche.